SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 1 – CO1

Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System

CO1: Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Field	Details	Field	Details
Student Name	C.Jayalakshmi	Register Number	192524052
Department / Section	AI&DS	Academic Year / Semester	2026
Faculty Name	Dr D. Antony Joseph Raja	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	2/11/2026	Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Introduce Cloud Computing	3		Definition; characteristics; deployment models; scenario mapping
2	Trace Evolution and Hardware Evolution	3		Timeline; hardware evolution; relevance to university
3	Analyse Internet Software Evolution	3		Web/software evolution; SOA; APIs; scenario relevance
4	Apply Server Virtualization	5		Hypervisor types; VM/container comparison; consolidation; limitations
5	Apply Web Services	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Analyse IaaS, PaaS and SaaS	5		Responsibility layers; five-application mapping; comparison; providers
7	Examine XaaS	3		Four or more variants; two justified choices;

				provider risks
8	Design Cloud Solution Architecture	5		Labelled diagram; deployment/service models; scaling; security; DR
9	Hands-on Implementation and Documentation	4		Running VM/container; PaaS/SaaS; tests; actual screenshots
10	Evaluate and Recommend Final Solution	7		Three-way comparison; 3-year costs; risks; phased roadmap
11	Day-4 Design Challenge	8		Impact; evaluation; adaptation; evidence-based defence
	TOTAL	**50**		**Percentage :**

EVIDENCE VERIFICATION CHECKLIST

Tasks	**Evidence expected**	**Status (✓ / P / ✗)**	**Evaluator note**
1–3	Conceptual explanations, comparisons and relevant illustrations	✓	Explanations and relevant scenario mapping documented
4	Virtualization calculations with stated assumptions and peak-load caution	✓	Calculations present; practical capacity validation recommended
5–7	API examples, service responsibility table and justified XaaS choices	✓	Required conceptual evidence documented
8	Labelled architecture diagram, caption below figure and component justification	✓	Architecture documented; sizing assumptions need testing
9	Actual code, environment details, execution logs, API outputs and screenshots	P	Procedures provided; actual execution evidence not established
10	Cost assumptions, 3-year totals, risk controls and migration roadmap	✓	Estimates and rationale documented
11	Day-4 changes, 10× load strategy, 99.9% target and 40-college isolation	✓	Design analysis documented; practical validation pending

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	**Observation**
Source code	✓	Flask REST API and Dockerfile provided in the proposed implementation
Virtualization	P	Docker container configuration documented; running-container evidence required
PaaS or SaaS	P	Managed web-service deployment procedure described; live deployment not verified
Functional test	P	Expected HTTP 200 response and JSON output documented; actual output required
Failure case	P	HTTP 404 test case documented; actual execution record required

Reproducibility	✓	Build commands, run commands, endpoints and expected results specified

_____ **Student Signature**	_____ **Faculty Signature**	_____ **Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Cloud definition, characteristics, deployment models and scenario mapping	✓	All conceptual requirements covered
2	Evolution timeline, hardware development and university relevance	✓	Well explained with relevant examples
3	Internet software evolution, SOA and APIs	✓	Clear explanation and scenario relevance
4	Hypervisors, VM/container comparison and consolidation calculations	✓	Calculations included; peak-load limitations discussed
5	SOAP/REST comparison and three university APIs	✓	API methods and request/response examples documented
6	IaaS/PaaS/SaaS responsibility layers and application mapping	✓	All five applications categorized with justification
7	XaaS variants and two justified selections	✓	Six variants explained; DBaaS and DRaaS recommended
8	Labelled architecture, scaling, security and disaster recovery	✓	Design and component justifications documented
9	Working container, PaaS/SaaS deployment, actual tests and screenshots	P	Code and procedures provided; actual execution evidence requires verification
10	Three-year cost comparison, risks and migration roadmap	✓	Cost assumptions, calculations and recommendations included
11	Impact analysis, 10× load, 99.9% availability and tenant isolation	✓	Changed requirements evaluated; load-test evidence recommended

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Source code	✓	Python Flask REST API and Dockerfile documented

Virtualization	P	Docker build and run procedures provided; execution screenshot required
PaaS or SaaS	P	Cloud deployment steps described; actual deployment confirmation required
Functional test	P	HTTP 200 and expected JSON response documented; actual test output required
Failure case	P	HTTP 404 test case included; execution evidence required
Screenshots	P	Required screenshots identified but authenticity and completeness not verified
Reproducibility	✓	Installation, configuration, commands and testing steps documented
Interpretation	✓	Technical outcomes and implementation limitations explained
Code and report consistency	✓	Proposed API, Docker configuration and testing procedure are aligned

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 11 tasks answered	☒ Yes ☐ Partly ☐ No
Cloud concepts and comparisons explained	☒ Yes ☐ Partly ☐ No
Virtualization calculations included	☒ Yes ☐ Partly ☐ No
API examples and service models explained	☒ Yes ☐ Partly ☐ No
Cloud architecture diagram included	☒ Yes ☐ Partly ☐ No
Implementation and testing evidence provided	☐ Yes ☒ Partly ☐ No
Three-year cost analysis and roadmap included	☒ Yes ☐ Partly ☐ No
Day-4 design challenge completed	☒ Yes ☐ Partly ☐ No
Figures and screenshots included	☐ Yes ☒ Partly ☐ No
Table and figure numbering correct	☐ Yes ☒ Partly ☐ No
Viva recommended	☒ Yes ☐ No

TASK 1 – INTRODUCE CLOUD COMPUTING AND MAP IT TO THE SCENARIO (3 MARKS)

1.1 Definition of Cloud Computing

Cloud computing is a model for delivering computing resources—such as servers, storage, databases, networks, software and processing power—over a network, usually the internet, whenever they are required. Instead of purchasing and maintaining every server inside the university, the institution can obtain resources from a cloud provider and pay according to usage or a selected subscription.

For the university, cloud computing means hosting its student records, learning platform, library services, examination portal and supporting databases on a scalable cloud environment. Students and staff can access these services through browsers and mobile applications from any campus or approved location.

1.2 Five Essential Characteristics

Characteristic	Explanation	Application to the university
On-demand self-service	Users or administrators can provision computing resources automatically without waiting for the provider's staff.	During admissions, the IT team can create additional application instances or storage when required.
Broad network access	Services are available over standard networks through devices such as laptops, phones and tablets.	Students at all four campuses can access results, course material and library services through a browser or mobile app.
Resource pooling	The provider pools computing resources and allocates them dynamically to multiple customers using a multi-tenant model.	The university shares a provider's large infrastructure while its own records remain logically isolated.
Rapid elasticity	Resources can be increased or decreased quickly according to demand.	Application capacity can increase during examination and result-publication peaks and reduce afterwards.

Measured service	Resource usage is monitored, measured and reported, enabling usage-based billing.	The university can monitor CPU, storage, database and network consumption and control its monthly cloud bill.

1.3 Cloud Deployment Models

Model	Meaning	Suitability for this university
Public cloud	Infrastructure is owned and operated by a cloud provider and offered to customers over the internet.	Useful for scalable web applications, managed databases, backups and public-facing APIs. It reduces capital expenditure.
Private cloud	Cloud infrastructure is dedicated to one organisation, either on-premises or hosted by a provider.	Can be selected for workloads requiring dedicated control, but it may retain high management and infrastructure costs.
Hybrid cloud	Combines private infrastructure with public cloud services, allowing controlled movement or integration of workloads and data.	A suitable transition model: sensitive legacy data can be migrated in phases while elastic application and backup services use public cloud.
Community cloud	Infrastructure is shared by organisations with common requirements, such as universities or government institutions.	Could support a consortium of colleges with shared academic services and common governance.

Recommended deployment model: a hybrid-cloud strategy during migration, moving towards a cloud-first architecture. Public-cloud services provide elasticity and managed services. Sensitive records are protected using encryption, access controls, private networking and approved data-residency arrangements. A private environment is retained only where a documented regulatory, technical or contractual requirement justifies it.

1.4 Three Existing Problems and Cloud-Based Solutions

Existing problem	Why it occurs	Cloud characteristic / solution
Under-utilised servers	Twelve dedicated servers average only 15% CPU utilisation, so capacity is wasted during normal periods.	Resource pooling and measured service consolidate workloads and make resource consumption visible.
Poor peak-period performance	CPU demand rises above 95% during admissions, results and examinations; fixed servers cannot expand quickly.	Rapid elasticity and auto-scaling add application instances when demand rises.
Slow procurement and no disaster recovery	Purchasing a server takes 6–8 weeks and the university has no secondary recovery site.	On-demand self-service provisions resources quickly; cloud backup and a geographically separate DR region improve recovery.

1.5 Benefits and Limitations

Benefits:

- Lower initial capital expenditure because the university does not need to purchase all new servers and storage in advance.
- Scalability during admissions, examinations and result publication, followed by scale-down during quieter periods.
- Improved availability through multiple availability zones, load balancing, backups and disaster-recovery services.
- Faster deployment of new services, including mobile applications and partner-college APIs.
- Centralised monitoring, automated patching options and managed databases can reduce routine administration.

Limitations and controls:

- Internet or provider outages can interrupt access; use redundant network links, multi-zone deployment and tested recovery procedures.
- Vendor lock-in may make migration difficult; prefer documented APIs, portable containers, open data formats and regular export tests.

- Data privacy and unauthorised access are risks; apply encryption, least-privilege IAM, audit logs, MFA and data classification.
- Usage-based charges can grow unexpectedly; configure budgets, alerts, tagging, quotas and periodic cost reviews.
- The university needs cloud-skilled staff; provide training, operating procedures and a phased migration.

Task 1 conclusion: a hybrid-cloud transition provides the best balance of flexibility, cost control and governance for the university's existing legacy environment.

TASK 2 – TRACE THE EVOLUTION AND HARDWARE EVOLUTION (3 MARKS)

2.1 Timeline of Computing Infrastructure

Stage	Main idea	Limitation addressed / contribution
Mainframe computing	A central, powerful computer serves many users through terminals.	Centralised processing replaced separate computing at every user location, but systems were expensive and highly centralised.
Minicomputers	Smaller and less expensive multi-user systems.	Made departmental computing more accessible than a single large mainframe.
Personal computers (PCs)	Processing moved to individual desktop machines.	Provided personal control and graphical interfaces, but created isolated data and management problems.
Client–server computing	Clients request services from central servers over a network.	Enabled shared databases and applications, but server capacity and administration remained fixed.
Cluster computing	Several connected computers work together as one service or compute system.	Improved performance and availability by combining multiple machines.

Grid computing	Distributed resources from different locations cooperate on large tasks.	Allowed sharing of geographically distributed computing capacity.
Utility computing	Computing resources are supplied and charged like a metered utility.	Introduced the pay-for-use concept and reduced the need to own all capacity.
Cloud computing	Virtualised, pooled resources and managed services are delivered on demand over networks.	Combines elasticity, self-service, broad access, measured usage and provider-managed infrastructure.

2.2 Hardware Evolution That Enabled Cloud Computing

Hardware development	**Technical contribution**	**Importance to cloud computing**
Multi-core processors	Several processing cores execute tasks in parallel within one processor.	Supports multiple virtual machines and concurrent application workloads.
Commodity servers	Standardised, relatively low-cost server hardware can be deployed in large numbers.	Providers can build large server farms economically instead of relying only on expensive specialised systems.
High-capacity storage	Large disk arrays, SSDs and distributed storage systems store huge volumes of data.	Supports student records, learning content, examination files, logs and backups.
High-speed networking	Faster Ethernet, fibre links and software-defined networking connect servers and storage.	Allows users and distributed cloud components to communicate with low delay and high throughput.

Large data centres	Facilities combine power, cooling, physical security, redundant networks and thousands of servers.	Provide the physical foundation for scalable and resilient cloud services.

2.3 How Each Stage Solved Earlier Limitations

The progression moved from centralised mainframes to departmental systems, then personal computers and networked client–server systems. Clusters and grids combined distributed computing resources, while utility computing introduced consumption-based access. Cloud computing built on these developments by using large data centres, virtualisation, automation and high-speed networks to provide resources on demand.

2.4 Current University System: Stage Identification

The university's current arrangement mainly represents traditional client–server computing with dedicated physical servers. Each application has its own server, and users connect to central services over the campus network or internet. It is not yet a mature cloud environment because resources are not pooled dynamically, provisioning takes 6–8 weeks, utilisation is low and there is no disaster-recovery site.

The university should move from this fixed client–server model towards virtualised, service-oriented and cloud-hosted infrastructure. This evolution directly addresses its under-utilised servers and inability to respond to sudden demand.

TASK 3 – ANALYSE INTERNET SOFTWARE EVOLUTION (3 MARKS)

3.1 Evolution of Internet Software

Stage	Characteristics	University relevance
Static web pages (early Web)	Mostly read-only HTML pages; content changes only when the page is manually updated.	Suitable for public notices but not sufficient for interactive student services.

Dynamic web	Server-side programs generate pages using databases and user input.	Enables login, student profiles, fee details and personalised dashboards.
Web 2.0	Interactive, participatory web applications with richer user interfaces and asynchronous updates.	Supports interactive LMS content, discussion forums, online forms and live status updates.
Service-Oriented Architecture (SOA)	Application functions are exposed as reusable, loosely coupled services with defined contracts.	SIS, fee payment and library functions can be reused by the portal, mobile app and partner colleges.
Mashups	Combines data or services from multiple sources through APIs.	A student dashboard can combine timetable, attendance, library due dates and fee status.
Cloud-hosted applications	Applications and supporting platforms are hosted on scalable cloud infrastructure and accessed through networks.	Makes university services accessible across campuses and supports elastic capacity.

3.2 Role of Browser, HTTP, AJAX and APIs

- Web browser: provides the user interface and sends requests to cloud-hosted applications. Students can use common browsers without installing a separate desktop program.
- HTTP/HTTPS: defines request and response communication between clients and web servers. HTTPS adds TLS encryption to protect credentials and personal information.
- AJAX: allows a web page to exchange data with a server asynchronously, commonly using JavaScript and JSON, without reloading the entire page. For example, a result panel can update after a student enters a register number.
- API (Application Programming Interface): defines how one application requests a function or data from another. A mobile app can call the university's result API instead of directly accessing the SIS database.

3.3 Desktop, Client–Server and Web-Delivered Software

Feature	Desktop software	Client–server software	Web-delivered software
Installation	Installed on each computer.	Client software may be installed; server is centrally maintained.	Usually accessed through a browser; minimal client installation.
Updates	Often need individual device updates.	Client and server updates may both be required.	Central server-side updates reach users immediately.
Access	Usually tied to a particular device or local network.	Available to clients connected to the server.	Available through internet or approved private network.
Scaling	Depends on each desktop's resources.	Requires server capacity planning.	Can use load balancing, cloud scaling and managed services.
Example	Standalone desktop library application.	A campus application connected to a central database server.	Cloud LMS or student portal accessed using HTTPS.

3.4 Effect on University Application Design

The university should gradually break tightly coupled monolithic applications into well-defined service modules. The user interface should be browser- and mobile-friendly, while business logic should be exposed through secure APIs. The database must not be accessed directly by mobile clients or partner colleges. An API gateway should authenticate callers, validate requests, apply rate limits and route calls to the correct service. This design improves reuse, maintainability and integration while allowing individual components to scale independently.

TASK 4 – APPLY SERVER VIRTUALIZATION (5 MARKS)

4.1 Working of a Hypervisor

A hypervisor, also called a virtual machine monitor (VMM), is the software layer that creates and manages virtual machines. It allocates physical CPU, memory, storage and network resources to each VM. Each VM receives virtual hardware and runs its own guest operating system and applications. The hypervisor schedules access to the physical hardware and helps isolate one VM from another.

4.2 Type-1 and Type-2 Hypervisors

Type	Operation	Examples	Suitability
Type-1 (bare-metal)	Runs directly on the physical server hardware without a general-purpose host operating system underneath.	VMware ESXi, Microsoft Hyper-V, Xen.	Recommended for university production servers because it is designed for server virtualisation, central management and performance.
Type-2 (hosted)	Runs as an application on top of a host operating system such as Windows, Linux or macOS.	Oracle VirtualBox, VMware Workstation.	Useful for student labs, demonstrations and development, but less suitable for the main production cluster.

4.3 Virtualisation Techniques

Technique	How it works	Advantages	Limitations / use
Full virtualisation	Hypervisor presents virtual hardware so an unmodified guest OS can run.	Good compatibility and strong isolation.	Can introduce some overhead; suitable for legacy SIS or exam systems that need a full OS.

Para-virtualisation	Guest OS is aware of the hypervisor and uses specialised interfaces or drivers for operations.	Can reduce overhead for supported workloads.	Requires guest OS or driver support; useful only where compatibility is confirmed.
OS-level virtualisation (containers)	Applications run in isolated user spaces while sharing the host OS kernel.	Lightweight, fast startup, high density and portable packaging.	Containers share the host kernel; use for stateless web/API services and microservices, not as a replacement for every VM.
Hardware-assisted virtualisation	CPU features such as Intel VT-x or AMD-V assist the hypervisor in running guest operating systems.	Improves support and efficiency for virtual machines.	Requires compatible CPU and firmware settings; commonly used with modern Type-1 hypervisors.

4.4 Proposed VM and Container Placement

Workload	Recommended environment	Reason
Legacy SIS application and database (where OS dependencies exist)	VMs initially; database may move later to managed DBaaS after compatibility testing.	VMs preserve the existing operating-system environment and provide a controlled migration path.
LMS web and application services	Containers or managed PaaS.	Web components can be scaled horizontally during course registration and learning peaks.
Library system	VM initially, then containers/PaaS if supported.	A VM reduces migration risk for a legacy application; a later modernised service can be containerised.

Exam portal	Isolated VMs or containers with dedicated resource limits and strong network policies.	Examinations need predictable capacity, strict access control and separation from non-critical workloads.
Email	SaaS email service.	Email is a standard service and is usually more efficient to consume from a specialist provider than to operate on university servers.

4.5 Server Consolidation Calculation

Given: 12 physical servers; average CPU utilisation per server = 15%; target utilisation of each new physical host = 70%. For this estimate, assume all 12 servers have equal CPU capacity and that CPU utilisation is a suitable first-order measure. The calculation excludes memory, storage IOPS, network throughput, licensing and peak-load headroom; these must be validated before procurement.

Step 1 – Calculate total current average CPU load:

Total load = Number of servers × Average utilisation

= 12 × 15% = 180% of one server's CPU capacity

This is equivalent to 1.8 fully utilised server-capacity units.

Step 2 – Calculate the number of hosts at the target utilisation:

Required hosts = Total load ÷ Target host utilisation

= 180% ÷ 70% = 2.57

Since a fraction of a physical host cannot be deployed, round up: minimum required hosts = 3.

Step 3 – Add availability and peak-load consideration:

Three hosts are the mathematical minimum for the stated average load. However, if one host fails, the remaining two would have to carry the load. The average load per two hosts would be 180% ÷ 2 = 90% of each host's capacity, above the 70% planning target. Therefore, a practical design should use at least

four hosts for N+1 resilience and additional headroom, subject to benchmarking. With four hosts, average load is 180% ÷ 4 = 45% per host.

Design	Host count	Average utilisation per host	Interpretation
Mathematical minimum	3	60% (180% ÷ 3)	Below 70% target in normal average conditions; no full N+1 capacity at the target.
Recommended initial cluster	4	45% (180% ÷ 4)	Allows one host to fail while three hosts carry about 60% average utilisation each.
Peak-load caution	Validate using measured peak CPU, memory and I/O	Not calculated from average alone	The source states peaks above 95%; admission and exam load tests are essential.

Important: the 4-host recommendation is a planning starting point, not a guaranteed final capacity. The university must collect peak CPU and memory metrics, benchmark each application, check database I/O and network demand, and run a host-failure test. If the 10× Day-4 demand is confirmed, the architecture must scale beyond this initial consolidation.

4.6 Virtualisation Benefits and Risks

Feature / risk	Explanation and university application
Live migration	Moves a running VM from one physical host to another with little or no user-visible interruption. It supports maintenance and load balancing.
Snapshots	Capture a VM's disk/state at a point in time and can assist rollback before a change. A snapshot is not a substitute for an independent backup.

Resource over-commitment	Allocates more virtual CPU or memory than physically available based on expected usage. It can improve utilisation but must be controlled to prevent contention.
Noisy neighbour	One VM or container consumes excessive CPU, memory, storage I/O or network bandwidth and affects other workloads. Use quotas, reservations, monitoring and workload separation.
Hypervisor failure	A host or hypervisor failure can affect all VMs on that host. Use a cluster, redundant hosts, backups and tested recovery.
Security isolation	A hypervisor vulnerability or incorrect network configuration could expose workloads. Patch regularly, restrict administration, segment networks and apply least privilege.

Task 4 conclusion: consolidate the 12 under-used servers using a Type-1 hypervisor cluster, initially plan for four hosts for N+1 resilience, run legacy workloads in VMs and use containers/PaaS for modern web services. Final sizing must be based on measured peak demand rather than average CPU alone.

TASK 5 – APPLY WEB SERVICES (4 MARKS)

5.1 Web Services and Their Role in SOA

A web service is a software function made available over a network through a defined interface and standard communication protocol. It allows independent applications to exchange data and request operations. In Service-Oriented Architecture (SOA), business functions are exposed as reusable, loosely coupled services. For example, the SIS can provide a student-record service that is used by the web portal, mobile application and authorised affiliated colleges.

5.2 SOAP, WSDL, UDDI and REST

Technology	Description	Typical use / distinction

SOAP	A protocol for structured message exchange, commonly using XML envelopes. It can support enterprise standards and formal messaging requirements.	Useful when strict contracts, formal message structure or enterprise WS-* features are required.
WSDL	Web Services Description Language; an XML document describing a SOAP service's operations, messages and endpoints.	Describes how a client can call a SOAP service; it is not itself a transport protocol.
UDDI	A directory standard intended for publishing and discovering web-service descriptions.	Conceptually supports service discovery; many modern systems use API catalogues and developer portals instead.
REST	An architectural style that uses resources and standard HTTP methods. JSON is common, though XML can also be used.	Well suited to mobile apps, web portals and partner integrations because it is lightweight and widely supported.

5.3 HTTP Methods Used by REST APIs

Method	Purpose	Example
GET	Read a resource; should not change server state.	GET /api/v1/students/192524052/results
POST	Create a resource or submit an operation.	POST /api/v1/fee-payments
PUT	Replace a resource or update it fully.	PUT /api/v1/students/192524052/contact
PATCH	Partially update a resource.	PATCH /api/v1/students/192524052/contact

DELETE	Delete a resource where permitted.	DELETE /api/v1/saved-searches/45

5.4 Three University Web Services with Examples

API 1 – Student Result Lookup

Purpose: allow an authenticated student to view their published examination results.

Endpoint: GET https://api.university.example/api/v1/students/{studentId}/results

Sample request: GET /api/v1/students/192524052/results

Sample JSON response:

{"studentId":"192524052","semester":5,"status":"PUBLISHED","results":[{"courseCode":"CSA15","courseName":"Cloud Computing","grade":"A"}]}

Choice: REST is suitable because the mobile app and web portal need a simple read operation and JSON is easy to process. The API must verify that the authenticated student is permitted to view the requested record.

API 2 – Fee Payment

Purpose: initiate a fee payment through an approved payment gateway. Card or bank credentials must be handled by the gateway, not stored by the university API.

Endpoint: POST https://api.university.example/api/v1/fee-payments

API 3 – Library Search

Purpose: search the library catalogue by title, author or keyword.

Endpoint: GET https://api.university.example/api/v1/library/search?q=cloud+computing

Sample response: {"query":"cloud computing","total":2,"items":[{"bookId":"B101","title":"Cloud Computing Concepts","availability":"AVAILABLE"},{"bookId":"B205","title":"Cloud Systems Design","availability":"ISSUED"}]}

Choice: REST is suitable because catalogue search is a read-heavy operation and can return compact JSON. Search results may be cached briefly, but live availability should be refreshed before a user places a hold.

5.5 When SOAP May Be Preferred

SOAP may be preferred for a legacy finance or government integration if the external system mandates WSDL contracts, XML schemas, WS-Security or a formal enterprise messaging standard. For the university's new mobile and partner-college APIs, REST over HTTPS is the recommended default because it is simpler to consume and supports common web technologies. SOAP and REST can coexist during migration.

Security requirements for all APIs: TLS encryption, OAuth 2.0 or OpenID Connect-based authentication, role- and scope-based authorisation, input validation, rate limiting, audit logging, versioning and monitoring. Never expose the internal database directly.

TASK 6 – ANALYSE IAAS, PAAS AND SAAS (5 MARKS)

6.1 Definitions

Infrastructure as a Service (IaaS) provides virtualised computing resources such as virtual machines, networks and storage. The provider manages the physical data centre and virtualisation infrastructure, while the customer manages the operating system, middleware, runtime, applications and data.

Platform as a Service (PaaS) provides a managed application platform, such as a runtime, application hosting environment or managed database. The provider manages the infrastructure and much of the platform; the customer focuses mainly on application code, configuration and data.

Software as a Service (SaaS) provides a complete application over the internet. The provider manages the application and underlying technology stack, while the customer manages users, access policies, configuration and its own data usage.

6.2 Responsibility by Layer

Layer	IaaS	PaaS	SaaS
Physical data centre, servers and storage	Provider	Provider	Provider
Networking and virtualisation	Provider	Provider	Provider
Operating system	Customer	Provider	Provider

Middleware and runtime	Customer	Provider	Provider
Application software	Customer	Customer (develop/configure)	Provider
Customer data and access configuration	Customer	Customer	Shared responsibility; customer governs its data and users

The exact division varies by provider and service. Even with SaaS, the university remains responsible for choosing appropriate access permissions, managing user accounts, classifying data and complying with institutional policies.

6.3 Mapping the Five University Applications

Application	Recommended model	Reason
Student Information System (SIS)	IaaS initially; modernise selected modules on PaaS later.	The SIS may contain legacy code and database dependencies requiring operating-system control during migration. IaaS supports a lift-and-shift first step; later refactoring reduces maintenance.

Learning Management System (LMS)	PaaS or managed LMS SaaS, depending on feature and integration needs.	A managed platform reduces server maintenance and can scale web workloads. SaaS is suitable if the product meets academic, accessibility and data-integration requirements.
Library System	IaaS initially, then PaaS/managed library SaaS if compatible.	A legacy catalogue and circulation system may need a controlled VM migration; a supported managed service can reduce long-term patching and backup effort.
Exam Portal	PaaS for the application tier, with managed database and isolated exam services.	Managed runtime and auto-scaling can handle examination peaks. Strong identity checks, audit trails, testing and dedicated resource limits are essential.
Email	SaaS.	Email is a standard commodity service. A specialist SaaS provider offers mailboxes, spam filtering, availability and collaboration features without the university maintaining mail servers.

6.4 Comparison of IaaS, PaaS and SaaS

Factor	IaaS	PaaS	SaaS

Control	Highest control over OS and application stack.	Control over application code and configuration; less control over runtime and OS.	Lowest technical control; configure the supplied application.
Cost	Usage-based infrastructure cost plus staff, licences and maintenance.	Usage-based platform cost; lower operating-system and middleware administration.	Subscription or per-user pricing; predictable for standard services but may rise with users/features.
Scalability	Can scale VMs and storage, but requires architecture and administration.	Often includes managed scaling and deployment features.	Provider scales the service; customer controls plan and usage limits.
Maintenance effort	High: customer patches OS, middleware and application.	Medium: provider manages platform; customer maintains application.	Lower technical maintenance; customer still manages users, data and configuration.
Vendor lock-in	Moderate; VM images and standard OSs can improve portability.	Potentially higher due to platform-specific runtimes and services.	Can be high because of proprietary data formats, workflows and integrations.

6.5 Provider Examples

- IaaS: Amazon Web Services EC2, Microsoft Azure Virtual Machines or Google Compute Engine.
- PaaS: Azure App Service, Google App Engine or AWS Elastic Beanstalk.
- SaaS: Microsoft 365 or Google Workspace for email and collaboration; a managed LMS product may also be consumed as SaaS.

Task 6 conclusion: use a mixed service-model strategy rather than forcing every application into one model. Keep legacy systems in IaaS during transition, use PaaS for modern web/API workloads and adopt SaaS for standard services such as email.

TASK 7 – EXAMINE XAAS (3 MARKS)

7.1 Meaning of XaaS

XaaS means Anything-as-a-Service. It is a broad approach in which technology capabilities are delivered as services over a network, usually through subscription or usage-based pricing. It allows an organisation to consume a specialised capability instead of building and operating the complete solution itself.

7.2 XaaS Variants

Variant	Meaning	University example
DaaS – Desktop as a Service	Hosted virtual desktops delivered to users over a network.	Provide controlled lab desktops to students and staff without managing a full desktop image on every device.
BaaS / MBaaS – Backend or Mobile Backend as a Service	Managed backend functions for mobile apps, such as authentication, notifications and data synchronisation.	Support the university mobile app with push notifications and managed identity integration.
FaaS – Function as a Service	Runs small event-triggered functions without managing servers.	Generate a notification after a result is published or process a payment gateway callback.
DBaaS – Database as a Service	Provider manages database provisioning, patching, backups and much of the database operation.	Host SIS or LMS databases on a managed database service after compatibility and security assessment.
Security as a Service	Security capabilities such as web application firewall, threat detection, identity protection and managed monitoring.	Protect public APIs and detect suspicious login or access patterns.
DRaaS – Disaster Recovery as a Service	Provides recovery infrastructure and orchestration for restoring workloads after a major outage.	Replicate critical SIS and examination data to a separate region and rehearse recovery.

7.3 Two Recommended XaaS Offerings

1. DBaaS: A managed relational database is recommended for suitable modernised applications. It can provide automated backups, patching options, monitoring, replication and managed failover. This reduces the university's database administration burden. Before migration, test database compatibility, latency, transaction performance, encryption, access control and export/restore procedures.

2. DRaaS: A geographically separate recovery environment is recommended because the current university has no disaster-recovery site. Critical databases and application configurations should be backed up or replicated according to defined recovery point objective (RPO) and recovery time objective (RTO). The university must test restoration regularly; merely enabling replication does not prove that the system can recover.

7.4 Risks of Depending on Multiple XaaS Providers

- Integration complexity: different identity, monitoring, billing and support systems can increase operational work.
- Vendor lock-in and data portability: proprietary databases, APIs or backup formats can make exit difficult.
- Security and privacy inconsistency: providers may have different access controls, locations and incident procedures.
- Service dependency: an outage or policy change at one provider can affect a connected university service.
- Cost accumulation: many small subscriptions and data-transfer charges may become difficult to forecast.

Mitigation: maintain a provider inventory, standardise identity and logging, use documented APIs, encrypt data, review contracts and exit clauses, export data periodically, and test recovery independently.

TASK 8 – DESIGN THE CLOUD SOLUTION ARCHITECTURE (5 MARKS)

8.1 Proposed Architecture

The proposed solution uses a hybrid-cloud migration approach. Public-facing access is delivered through a secure edge and load balancer. Web and API workloads run on a scalable application tier.

Data is stored in managed databases and object storage. Critical services are deployed across multiple availability zones, with backups and a recovery copy in a separate region.

8.2 Component and Service-Model Selection

Component	Deployment / service model	Purpose and justification
DNS, WAF and load balancer	Public cloud managed service / security service	Routes requests, filters common web attacks and distributes traffic across healthy instances.
Web and API gateway tier	PaaS or container platform	Provides secure entry points, API versioning, authentication integration, rate limiting and routing.
Legacy SIS and library services	IaaS virtual machines initially	Preserves compatibility while the university assesses and modernises legacy code.
New application services	PaaS or container orchestration	Allows independent deployment and horizontal scaling of modules.
Database tier	DBaaS in private network	Provides managed database operations, backups and high-availability options.
Documents, images and backups	Object storage	Stores learning content, receipts, logs and backup copies cost-effectively.
Email	SaaS	Avoids operating a separate university mail-server infrastructure.
Disaster recovery	DRaaS / secondary cloud region	Provides a geographically separate recovery copy and planned failover capability.

8.3 Elasticity and Peak Handling

The load balancer distributes incoming requests across healthy application instances. Auto-scaling policies monitor metrics such as CPU, request rate, queue length and response time. When demand rises during admissions or examinations, additional instances are launched; when demand falls, excess capacity is removed. Scheduled scaling can be used for known academic events, while metric-based scaling handles unexpected spikes. Database scaling requires separate planning because adding web instances does not automatically increase database transaction capacity.

8.4 Security and Availability Measures

- Identity and access: central identity provider, multi-factor authentication for administrators, role-based access control and least privilege.
- Network security: private subnets for databases, security groups/firewall rules, WAF at the public edge and no direct public database access.
- Encryption: HTTPS/TLS in transit and encryption at rest for databases, object storage and backups; protect keys using a managed key service.
- Data protection: automated backups, point-in-time recovery where supported, retention policies and periodic restore tests.
- Availability: deploy critical services across at least two availability zones, use health checks and avoid single points of failure.
- Monitoring: central logs, metrics, alerts, audit trails and incident-response procedures.
- Application security: secure coding, dependency scanning, patching, input validation and regular vulnerability testing.

8.5 Disaster Recovery Planning

The university should define an RPO (maximum acceptable data loss measured in time) and an RTO (target time to restore service) for each application. For example, fee transactions and examination records should have stricter recovery targets than public informational pages. Backups should be encrypted and stored separately from the production account or region. A recovery runbook must identify decision-makers, restore order, DNS changes, data validation and communication to students. The numerical RPO/RTO targets must be approved by university management after a business-impact analysis.

TASK 9 – HANDS-ON IMPLEMENTATION AND DOCUMENTATION (4 MARKS)

Important evidence note: the assignment rubric specifically requires an actually running VM/container, a deployed PaaS or SaaS component, real test output and authentic screenshots. The steps and sample outputs below are a reproducible proof-of-concept guide; they must not be presented as screenshots or execution results unless the student runs them and captures the real output.

9.1 Proof-of-Concept Objective

Build a small Student Result REST API using Python Flask, package it in a Docker container and test it using a browser, Postman or curl. This demonstrates the application/API tier and container-based virtualisation. For the PaaS element, deploy the same small application to a managed application-hosting platform or use an approved cloud-hosted static site/database service. Record the actual provider, region, plan and deployment URL in the final submission.

9.2 Example Project Structure

university-cloud-poc/
|-- app.py
|-- requirements.txt
|-- Dockerfile

9.3 Flask REST API Code

```
# app.py
from flask import Flask, jsonify

app = Flask(__name__)

RESULTS = {
   "192524052": {
      "studentId": "192524052",
      "semester": 5,
      "status": "PUBLISHED",
      "results": [
         {"courseCode": "CSA15", "courseName": "Cloud Computing",
```

```
            "grade": "A"}
        ]
    }
}

@app.get("/")
def home():
    return jsonify({
        "service": "University Student Result API",
        "status": "running"
    })

@app.get("/api/v1/students/<student_id>/results")
def get_results(student_id):
    record = RESULTS.get(student_id)
    if record is None:
        return jsonify({"error": "Student record not found"}), 404
    return jsonify(record), 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

This is demonstration data only. A real university deployment must use an authenticated database-backed service and must not expose student records through an unauthenticated endpoint.

9.4 requirements.txt

```
Flask>=3.0,<4.0
```

9.5 Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

9.6 Build and Run the Container

1. Install Docker Desktop and ensure the Docker engine is running.
2. Open Command Prompt or the VS Code terminal in the university-cloud-poc folder.
3. Create the files app.py, requirements.txt and Dockerfile using the exact contents above.
4. Build the image: docker build -t university-result-api .
5. Run the container: docker run --name university-api -p 5000:5000 university-result-api
6. Open http://127.0.0.1:5000/ in a browser. The root endpoint should return a JSON service-status message.
7. Test a valid student: curl http://127.0.0.1:5000/api/v1/students/192524052/results
8. Test an unknown student: curl -i http://127.0.0.1:5000/api/v1/students/UNKNOWN/results

9.7 Expected Test Format (Compare with Actual Output)

Successful request: HTTP 200 OK with JSON containing studentId, semester, status and results. Unknown student: HTTP 404 NOT FOUND with {"error":"Student record not found"}. These are expected results from the example code, not evidence that the program has already been executed.

9.8 PaaS Deployment Procedure

9. Choose an approved PaaS provider and create a web application/service.
10. Connect the source repository or upload the project according to the provider's instructions.
11. Set the build command to install requirements and the start command to run the Flask application using a production WSGI server (for example, gunicorn app:app).
12. Configure the platform's required port and environment variables; do not hard-code secrets.
13. Deploy the service and copy the actual HTTPS URL.
14. Test the root and result endpoints using a browser or Postman. Record status codes, response bodies, date/time and region.
15. Configure authentication before using real student data. Use synthetic data only for this assignment demonstration.

Figure 2 – Docker build	Terminal showing the actual successful docker build command and image creation.

Figure 3 – Running container	docker ps output showing the university API container running and port mapping.
Figure 4 – API success	Browser/Postman/curl showing actual HTTP 200 response for the sample student.
Figure 5 – API failure	Actual HTTP 404 response for an unknown student.
Figure 6 – PaaS deployment	Provider dashboard showing deployment status and service name; hide account secrets.
Figure 7 – Hosted endpoint	Actual HTTPS endpoint response from the deployed service.

Do not insert fabricated screenshots. If a cloud free tier requires a payment method or has changed its limits, use a permitted local Docker/VirtualBox demonstration and clearly state that PaaS deployment remains pending.

TASK 10 – EVALUATE AND RECOMMEND THE FINAL SOLUTION (7 MARKS)

10.1 Three-Way Solution Comparison

Factor	Existing on-premises	IaaS-based migration	PaaS/SaaS-oriented solution
Cost model	High CapEx for servers, storage, refresh and data-centre facilities; ongoing staff and power costs.	Lower initial CapEx; usage-based VM, storage and network charges plus OS and application administration.	More OpEx through subscriptions/managed usage; less infrastructure administration, but recurring service charges.

Scalability	Limited by installed hardware and procurement delay of 6–8 weeks.	Can provision VMs faster, but OS and capacity management remain the university's responsibility.	Managed scaling can be simpler for supported workloads; quotas and database limits still need planning.
Availability	Single data centre and no DR site create a major single-site risk.	Can deploy across zones and create a separate recovery environment, if designed and paid for.	Provider-managed resilience may be available, but service-specific SLA, dependencies and recovery options must be checked.
Security	Direct physical control, but university must maintain all security layers and facilities.	More control over OS and network; university remains responsible for patching and configuration.	Provider manages more layers; university remains responsible for identity, permissions, data governance and configuration.
Management effort	High hardware, OS, backup and facility maintenance.	Medium to high; cloud infrastructure is flexible but still needs skilled administrators.	Lower infrastructure effort, but integration, vendor governance and contract management remain.

10.2 Illustrative Three-Year Cost Comparison

The assignment does not provide actual vendor quotations, server purchase prices, staffing costs, database sizes, network usage or subscription rates. Therefore, the following is an illustrative planning estimate in Indian rupees, not a provider quote or a guaranteed university budget. It is included to demonstrate a transparent comparison. Actual procurement must replace these assumptions with quotations and measured usage.

Cost assumption	On-premises	IaaS migration	PaaS/SaaS mix
Initial migration / setup	₹0.0 lakh	₹8.0 lakh	₹10.0 lakh
Annual infrastructure / service charges	₹18.0 lakh	₹15.0 lakh	₹17.0 lakh
Annual operations / support	₹8.0 lakh	₹6.0 lakh	₹4.0 lakh
Three-year recurring total	₹78.0 lakh	₹63.0 lakh	₹63.0 lakh
Three-year total including setup	₹78.0 lakh	₹71.0 lakh	₹73.0 lakh

Working:

On-premises = 3 × (₹18 lakh + ₹8 lakh) = ₹78 lakh.

IaaS = ₹8 lakh setup + 3 × (₹15 lakh + ₹6 lakh) = ₹71 lakh.

PaaS/SaaS = ₹10 lakh setup + 3 × (₹17 lakh + ₹4 lakh) = ₹73 lakh.

Interpretation: under these assumptions, IaaS has the lowest three-year total, while PaaS/SaaS has lower annual operations effort. The figures do not include every possible tax, bandwidth, data-egress, licence, premium support or growth charge. A sensitivity analysis should be performed for the 10× load requirement.

10.3 Main Risks and Mitigation

Risk	Potential impact	Mitigation
Vendor lock-in	Migration to another provider may be expensive or slow.	Use open standards, containers where suitable, documented APIs, portable backups and periodic exit tests.

Data privacy	Student records or payment data may be exposed or processed in an unsuitable location.	Classify data, review contracts and applicable law, encrypt data, apply least privilege, audit access and minimise stored payment data.
Cloud downtime	Students may be unable to access results, fees or examinations.	Multi-zone architecture, health checks, backups, DR region, tested runbooks and provider SLA review.
Skills gap	The university may misconfigure or poorly operate cloud resources.	Train staff, use infrastructure-as-code, document operations, conduct security reviews and engage support during migration.
Unexpected cost growth	Autoscaling, storage, data transfer or logging can exceed budget.	Budgets, alerts, resource tagging, quotas, lifecycle policies, reserved capacity where appropriate and monthly cost reviews.

10.4 Phased Migration Roadmap

Phase	Activities	Reason / exit condition
Phase 1 – Assess and prepare	Inventory applications, dependencies, data classification, network links, peak metrics, RPO/RTO and licensing.	Creates a verified baseline and avoids moving incompatible workloads blindly.
Phase 2 – Foundation	Set up cloud accounts, IAM, network segmentation, logging, encryption, budgets and backup policies.	Establishes secure governance before production migration.

Phase 3 – Low-risk pilot	Move a non-critical web page or library search component; test container and PaaS deployment.	Validates skills, network connectivity, monitoring and deployment process.
Phase 4 – Email and collaboration	Adopt SaaS email after identity, domain, retention and migration testing.	Email is a standard service and reduces on-premises maintenance.
Phase 5 – LMS and library	Move suitable components to PaaS/containers; retain a VM for components that cannot yet be modernised.	Moderate business risk and useful opportunity to validate scaling.
Phase 6 – SIS and fee services	Migrate in controlled waves with data validation, API gateway, rollback plan and reconciliation.	These are critical systems; migrate only after security and transaction tests pass.
Phase 7 – Exam portal and optimisation	Deploy isolated exam workloads, load test, conduct DR exercise and tune autoscaling.	Protects high-impact exam periods and confirms capacity before production use.

10.5 Final Recommendation

The university should adopt a phased hybrid-cloud migration, using IaaS for legacy systems that cannot immediately be changed, PaaS and containers for newly developed web/API services, SaaS for email, and DBaaS/DRaaS where compatibility and governance checks are satisfied. This approach avoids a risky single-step replacement, improves scalability and creates a recovery capability absent from the current data centre. The university should select a provider only after comparing service-level commitments, data location, security controls, support, portability and total cost of ownership.

TASK 11 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

11.1 Impact Analysis (2 Marks)

The changed requirement introduces a mobile application, exposes student-record and fee-payment services to 40 affiliated colleges, increases expected demand to 10 times the current peak and requires 99.9% availability under a limited operating budget.

Original assumption	Changed condition	Design impact
Mainly four-campus users (25,000 students and 2,000 staff)	40 affiliated colleges also consume selected services.	Identity, API governance, tenant separation, onboarding and audit requirements become critical.
Peak load already exceeds 95% on fixed servers	Expected load is 10× the current peak.	A fixed VM cluster sized only from average CPU is insufficient; horizontal scaling and load testing are required.
No explicit mobile/partner API requirement	Mobile app and partner-facing student/fee APIs are required.	A versioned API gateway, developer documentation, authentication and rate limits are essential.
Single data centre; no DR site	99.9% availability target.	The design needs multi-zone deployment, tested backups and a recovery strategy.
Internal university user access	Multi-tenant access by 40 colleges.	Each college must access only its authorised students and transactions; tenant isolation must be enforced at every layer.

API importance: the mobile app and college systems should not connect directly to SIS databases. APIs provide a controlled contract, authentication, validation, rate limiting, versioning, monitoring and

consistent business rules. This reduces integration risk and allows the university to change internal systems without breaking every client.

11.2 Evaluation of the Original Design (2 Marks)

The original mixed IaaS/PaaS/SaaS plan is a sound migration foundation, but it is not sufficient without modification. A simple four-host consolidation plan based on average CPU load cannot be assumed to handle 10× peak demand. The university must benchmark realistic traffic, database concurrency, storage I/O and network throughput. Auto-scaling helps the stateless web/API tier, but it cannot automatically remove bottlenecks in a single database or a third-party payment gateway.

A single cloud region is not sufficient for a robust 99.9% design if a regional outage is within the university's availability-risk model. At 99.9% availability, the maximum theoretical downtime in a 365-day year is:

$365 \times 24 \times 60 \times (1 - 0.999) = 525.6$ minutes = 8.76 hours per year.

This is an error budget, not a promise that the system will achieve the target. The university should deploy across multiple availability zones and maintain a tested secondary-region recovery plan. The final SLA depends on the provider's service commitments and the university's own application design.

11.3 Adaptation / Redesign (2 Marks)

Decision: MODIFY the original design rather than retain it unchanged or replace it completely. The original hybrid-cloud foundation remains useful, but the public access, API, application, database, tenant isolation and recovery layers must be strengthened.

Revised architecture changes

- Place a WAF and API gateway in front of all mobile and partner-college traffic. Require OAuth 2.0/OpenID Connect tokens, scopes, rate limits, request validation and API versioning.
- Use a container orchestration platform or managed PaaS for stateless services such as student lookup, fee status and notifications. Autoscale based on request rate, latency, CPU and queue depth.
- Separate tenant context for all 40 colleges. Every request must carry a verified tenant identity; enforce tenant-aware authorisation in the service and database layer, not only in the user interface.
- Use database row-level security or a tenant-key strategy with strict query filtering. For high-sensitivity workloads, consider separate schemas or databases for stronger isolation, based on cost and compliance assessment.
- Keep payment processing with a certified payment gateway. Use idempotency keys, signed callbacks, reconciliation and minimal storage of payment information.

- Deploy application instances across at least two availability zones. Use health checks, redundant load balancers and managed database high availability.
- Maintain encrypted backups and a secondary-region recovery copy. Define RPO/RTO for each service and conduct scheduled failover and restore drills.
- Use caching for safe, non-sensitive reference data and CDN delivery for static content. Do not cache private student records unless the cache is correctly authenticated, scoped and controlled.

Revised service-model mix

Component	Revised choice	Reason
Legacy SIS	IaaS VM initially, with APIs in front	Avoids a risky rewrite while controlling access through a modern API layer.
Mobile and partner APIs	API gateway + PaaS/container services	Supports independent scaling, authentication, throttling and version management.
Student/fee microservices	PaaS or orchestrated containers	Stateless services can scale horizontally for a 10× traffic spike.
Database	Managed DBaaS with high availability and tenant controls	Reduces administration while supporting backups, monitoring and controlled failover.
Notifications and event processing	FaaS / MBaaS where suitable	Event-triggered functions can handle result notifications and payment callbacks without always-on servers.
Email	SaaS	Continues to reduce operational burden.
Recovery	DRaaS / secondary region	Provides geographically separate recovery capability.

11.4 Evidence-Based Defence (2 Marks)

A. Capacity Calculation for 10× Demand

The question states that expected load is 10 times the current peak, but it does not provide the actual peak CPU quantity, request rate, transaction rate or VM host capacity. Therefore, an exact host count cannot be calculated honestly from the supplied data. The correct sizing method is to benchmark the current peak and multiply the measured demand by ten.

Let measured current peak demand = P capacity units. New peak demand = 10P. If one application instance safely handles C capacity units at the required response time, and the target maximum utilisation is 70%, then:

Required application instances = ceiling(10P ÷ (0.70 × C)).

For illustration only, if a load test measures 100 requests/second at the current peak and one instance safely handles 50 requests/second at the target latency, then new demand = 1,000 requests/second. Required instances = ceiling(1,000 ÷ (0.70 × 50)) = ceiling(28.57) = 29 instances. This is an example calculation, not a measured university result. Database capacity, network throughput and external payment limits must be load-tested separately.

B. Availability Design

A 99.9% target permits approximately 8.76 hours of downtime per year. Multi-zone deployment reduces the risk of a single data-centre fault, but the university must also remove application and database single points of failure. A secondary region, tested backups, documented failover and monitoring are required for regional incidents. The university should define whether planned maintenance counts against its availability target and how availability is measured.

C. Multi-Tenant Security

- Authenticate each user and identify the college/tenant from a trusted identity claim, not from an unverified request parameter.
- Authorise each operation using tenant ID, role and resource ownership. A college administrator must not read another college's student records.
- Apply tenant-aware database access controls and automated tests for cross-tenant data leakage.
- Encrypt data in transit and at rest; restrict support staff access and log privileged actions.
- Apply per-tenant API quotas and anomaly detection so one college cannot exhaust shared capacity.
- Keep payment operations traceable and reconcile payment gateway notifications against university invoices.

D. Original vs Revised Cost

A precise cost cannot be calculated without provider pricing and measured traffic. The revised design is expected to cost more than the original design because it adds API gateway usage, additional application instances, managed database capacity, multi-zone redundancy, monitoring and secondary-region recovery. To keep the operating budget limited, use autoscaling and scale-to-zero/serverless functions for suitable event workloads, apply caching to safe content, reserve predictable baseline capacity where economical, set per-tenant quotas and review data-egress charges.

Cost area	Original design	Modified design
Compute	Small VM/container capacity for current campus load.	Autoscaling application tier sized from measured 10× peak; scale down outside peak periods.
API management	Basic internal APIs.	API gateway, authentication, throttling, logging and versioning for mobile and 40 colleges.
Database	Single managed database or initial VM database.	High-availability DBaaS, performance tiering, tenant controls and tested recovery.
Availability / DR	Multi-zone and backup design planned.	Multi-zone production plus secondary-region recovery and regular failover tests.
Security / monitoring	Core IAM, encryption and logs.	Tenant-level policies, API threat protection, per-tenant monitoring and stronger audit coverage.

E. Trade-offs

The modified design improves scalability, isolation and resilience but increases architectural complexity, provider dependency and operating cost. Container orchestration and multi-region recovery require skilled administrators. Managed services reduce some maintenance but may increase lock-in. The university should therefore use managed services where they provide measurable operational value, keep portable application containers and data exports, and migrate in phases with rollback plans.

Task 11 conclusion: modify the original hybrid-cloud architecture. Retain the phased migration foundation, but introduce a secure API gateway, horizontally scalable application services, tenant-aware authorisation, high-availability database services, multi-zone deployment and secondary-region disaster recovery. Validate the design with 10× load tests and tenant-isolation tests before opening services to all 40 colleges.

CONCLUSION

Cloud computing provides the university with a practical path from fixed, under-utilised physical servers to a scalable and measurable digital platform. The recommended approach is a phased hybrid-cloud migration: IaaS protects legacy compatibility during transition, PaaS and containers support modern web and API workloads, SaaS replaces standard services such as email, and DBaaS/DRaaS improve database operations and recovery. The initial consolidation estimate is three hosts mathematically, with four hosts recommended as a more resilient starting point; however, peak-load testing is essential because the current environment exceeds 95% utilisation during critical periods. Under the Day-4 requirement, the architecture must be modified to support 10× demand, 40-college tenant isolation and 99.9% availability. Final acceptance should depend on measured performance, security testing, cost validation and a successful disaster-recovery exercise.

urement.